SDR

Estampilles

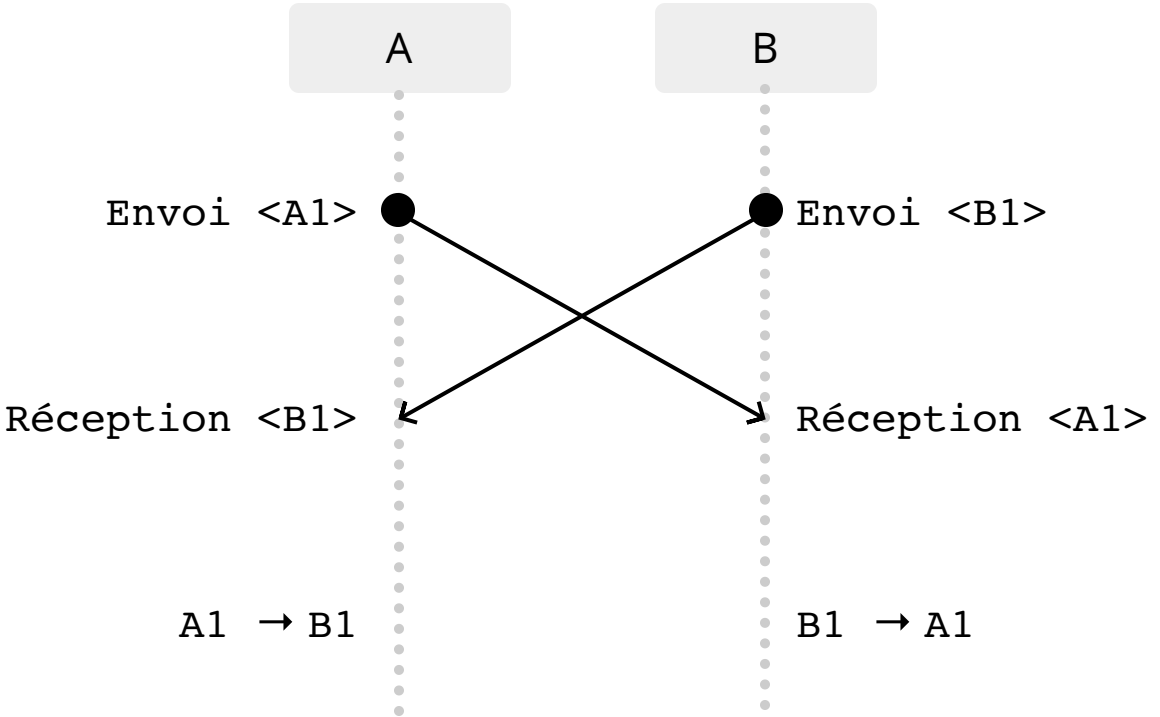
Horloges logiques

Olivier Lemer

↓ Le problème des horloges

Le problème des horloges

L'ordre des évènements n'apparait pas le même partout.



La synchronisation comme solution ?

Créer une horloge commune ?

Problème : Horloges à quartz dérivent différemment selon la machine.

Solutions : protocoles de mise à l'heure.

- Network Time Protocol : longue distance, ms precision.
- Precision Time Protocol : courte distance, µs precision.

Pas de solution permettant une horloge globale parfaite :
toujours soumis aux délais et variabilités du réseau.

↓ Nos besoins

Résolution par requirements

Quels sont les besoins ?

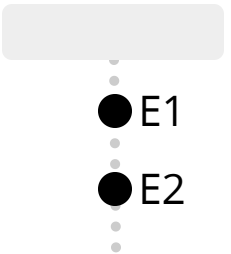
On veut connaitre ~~l'heure~~ ~~exacte~~ ~~de chaque évènement.~~
~~relative~~
l'**ordre** des évènements.
? **?**

Qu'est-ce qu'un évènement ?

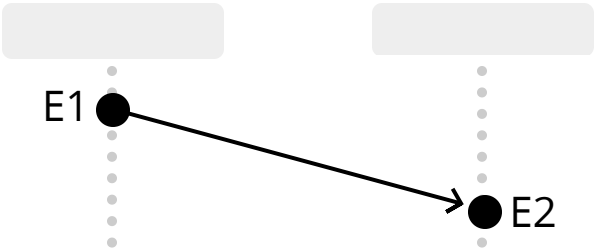
- Une action sur une machine *(exécution d'une ou plusieurs instructions)*
- Emission d'un message
- Réception d'un message

Qu'est-ce qu'un ordre d'évènements ? *Quand est-ce que E1 → E2 ?*

- Si E1 et E2 sont sur la même machine et E1 arrive avant E2.



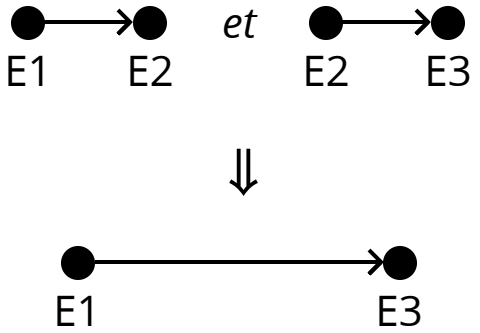
- Si E1 est l'émission d'un message et E2 sa réception.



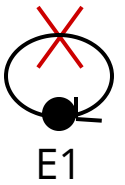
Une relation d'ordre

Propriétés

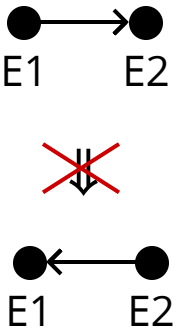
Transitif



Anti-réflexif



Anti-symétrique



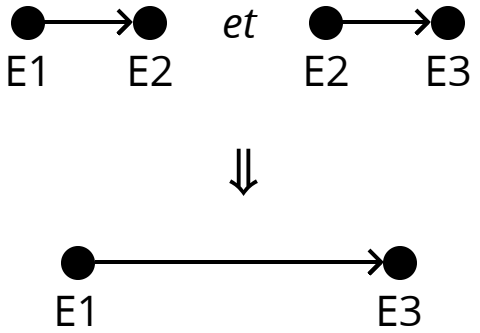
Ordre Partiel ou Total ?

(Partiel s'il existe deux évènements E1 et E2 tels que ni E1 → E2, ni E2 → E1.)

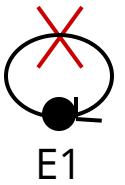
Une relation d'ordre

Propriétés

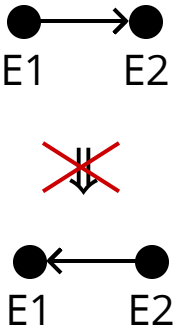
Transitif



Anti-réflexif

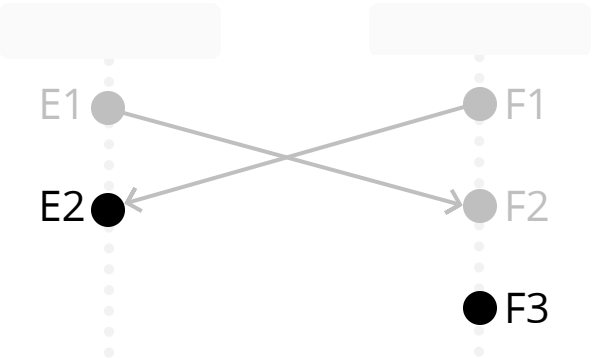


Anti-symétrique



Ordre Partiel

(i.e. il existe deux évènements E1 et E2 tels que ni E1 → E2, ni E2 → E1.)



E2 → F3 ?

F3 → E2 ?

Les deux sont possibles.
E2 et F3 n'ont pas d'ordre.
E1 et F1, E2 et F2, non plus.

↓ Horloge logique

Une horloge *logique*

Représenter l'ordre partiel des évènements.

On veut : une fonction H telle que :

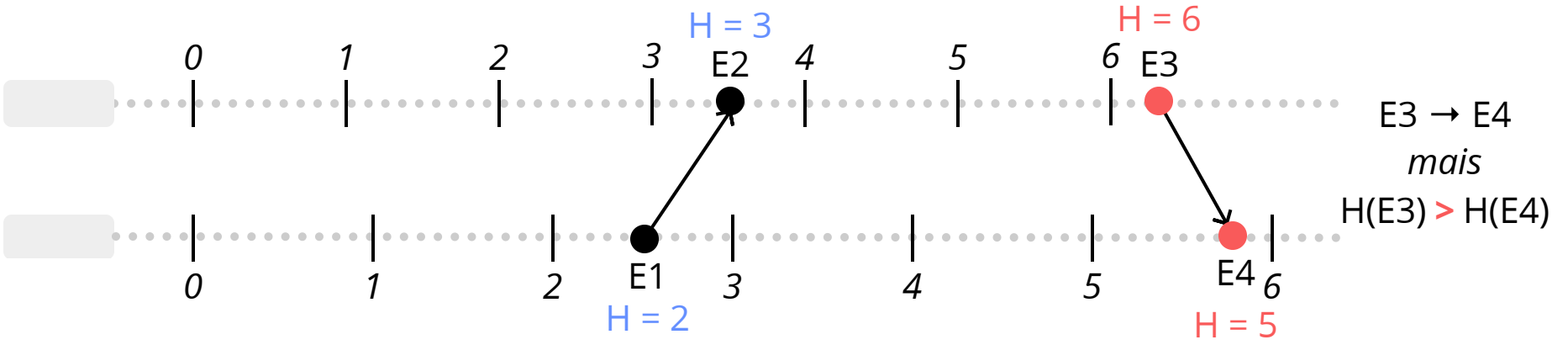
$$E1 \rightarrow E2$$

$$\Downarrow$$

$$H(E1) < H(E2)$$

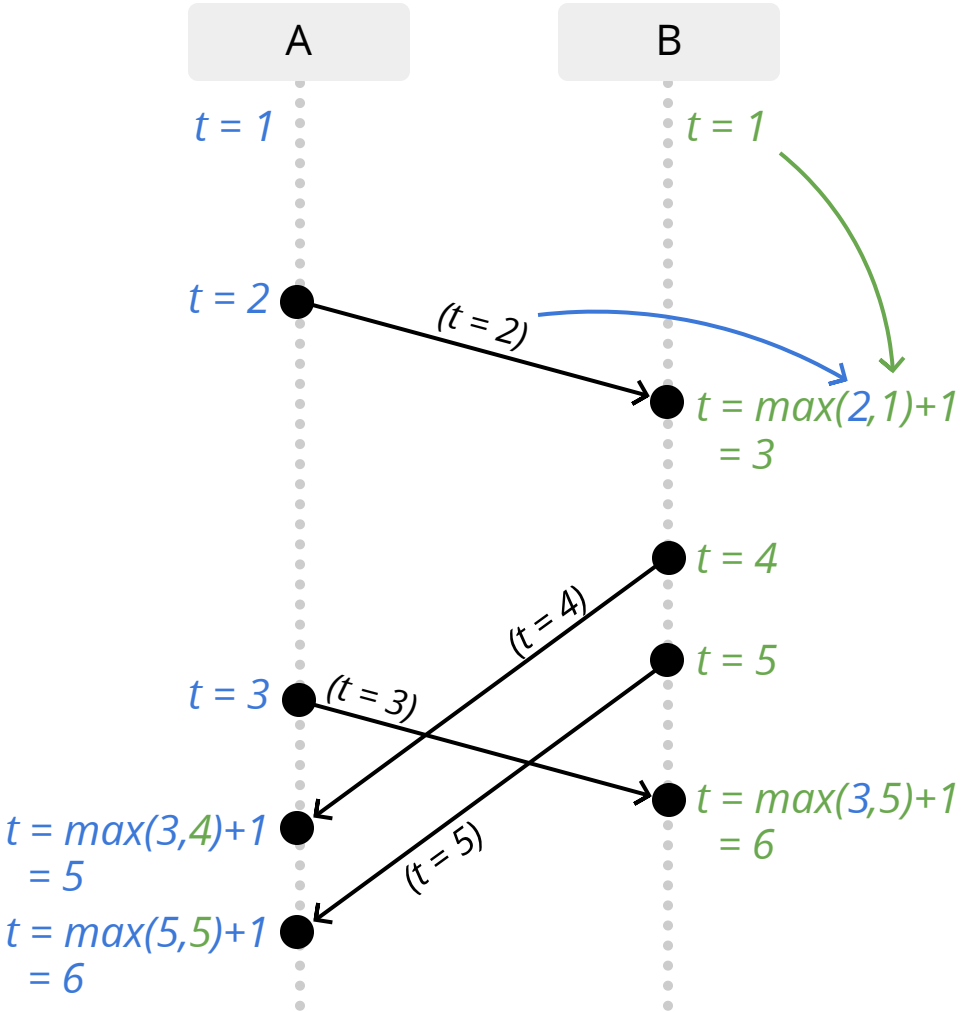
Idée 1 : horloge logique = horloge locale ?

(i.e. H(E) = heure sur la machine sur laquelle l'évènement a eu lieu)



L'horloge logique *de Lamport*

L'idée : les timestamps *(ou estampilles)*



- Chaque site maintient un numéro.
- À chaque événement *local*, le timestamp est incrémenté.
- Le timestamp est envoyé avec chaque nouveau message.
- À la réception, le timestamp le plus grand entre local et reçu est utilisé *et incrémenté*.

L'horloge logique *de Lamport*

Vérification des besoins

On voulait : une fonction H telle que :

$$E1 \rightarrow E2$$
$$\Downarrow$$
$$H(E1) < H(E2)$$

On avait défini $E1 \rightarrow E2$:

- Si E1 et E2 sont sur la même machine et E1 arrive avant E2.

$$\Downarrow$$
$$H(E1) < H(E2)$$



- Si E1 est l'émission d'un message et E2 sa réception.

$$\Downarrow$$
$$H(E1) < H(E2)$$



L'horloge logique *de Lamport*

Vérification des besoins - Un ordre total ?

On voulait : une fonction H telle que :

$$E1 \rightarrow E2$$
$$\Downarrow$$
$$H(E1) < H(E2)$$

Remarque :

$$E1 \rightarrow E2$$
$$\not\Uparrow$$
$$H(E1) < H(E2)$$

*On projette un **ordre partiel** sur un **ordre total**.*
Autrement dit, si E1 et E2 n'ont pas de relation, H leur en donne une.

L'horloge logique *de Lamport*

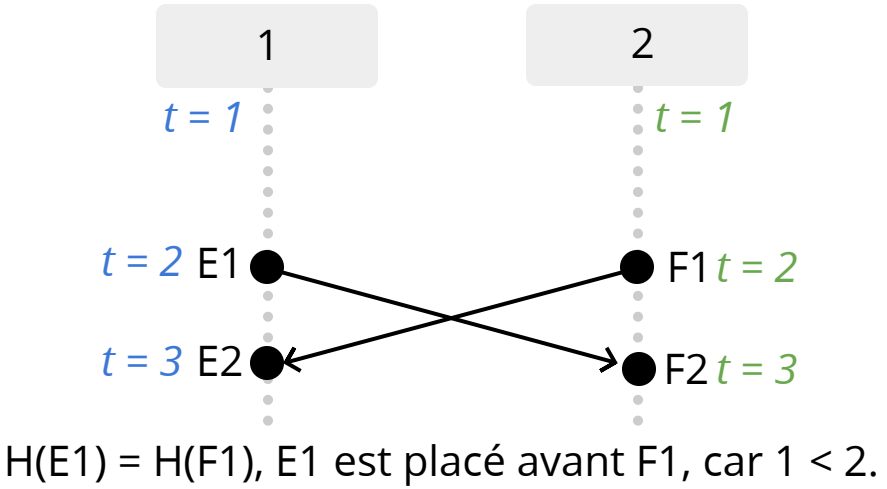
Vérification des besoins - Un ordre total *strict ?*

Remarque : On peut avoir H(E1) = H(E2).
*Ce n'est pas un ordre **strict**.*

Compliqué pour prendre des décisions, affecter des priorités, etc.

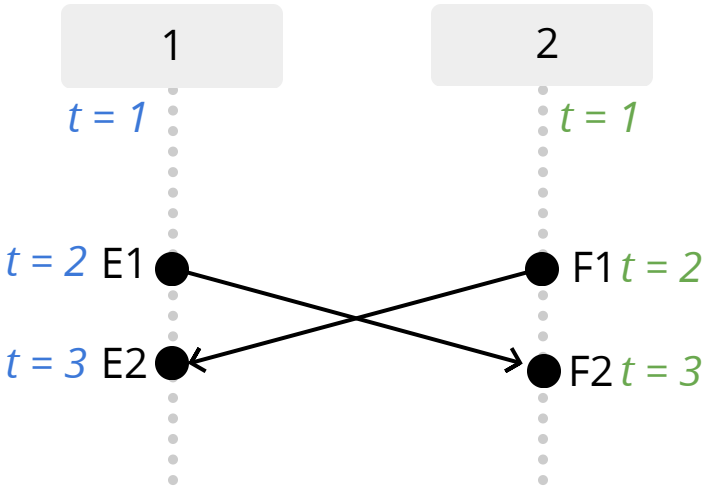
Solution : le rendre strict, arbitrairement :
priorité à la machine à l'ID le plus petit.

Si E1 arrive sur machine `i`,
E2 arrive sur machine `j`,
tels que `i` < `j`
et H(E1) = H(E2),
Alors E1 est arrivé avant E2.



L'horloge logique *de Lamport*

Ordre total rendu strict ; Exemple

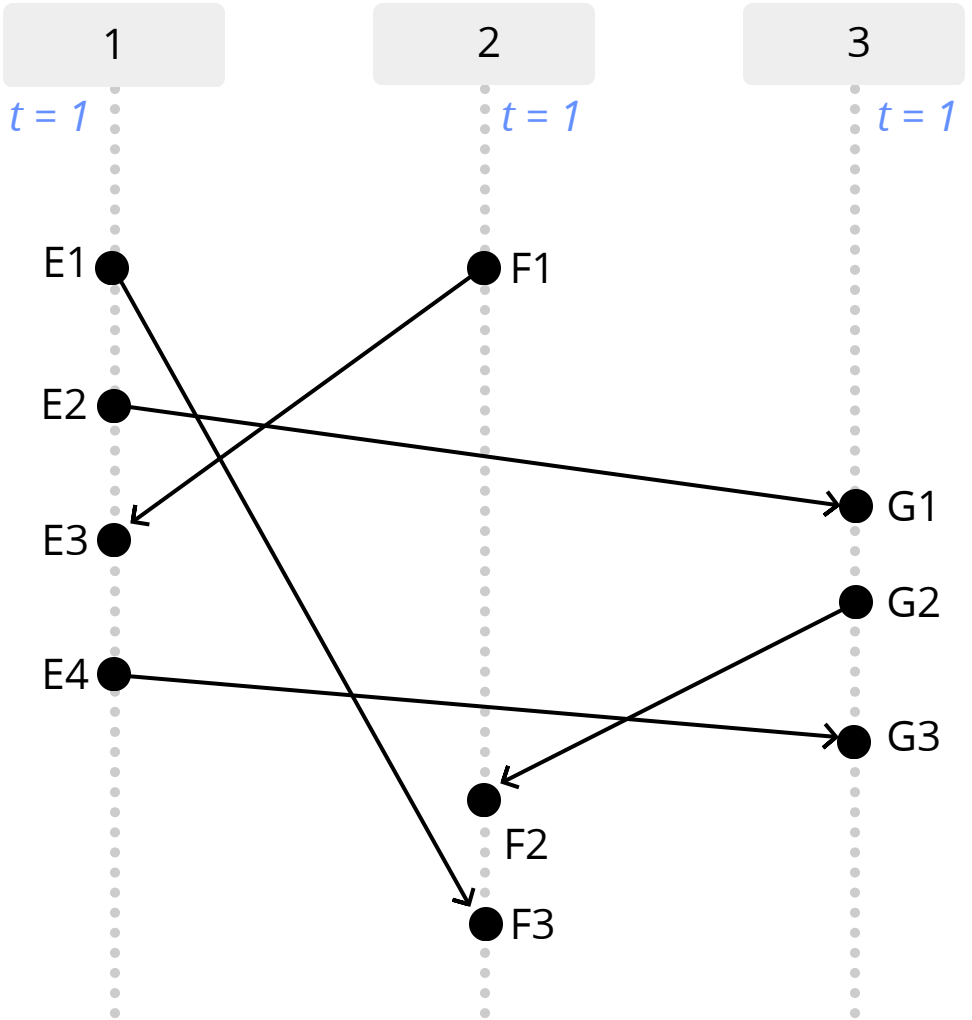


Quel est l'ordre ?

E1, F1, E2, F2.

↓ Exercices

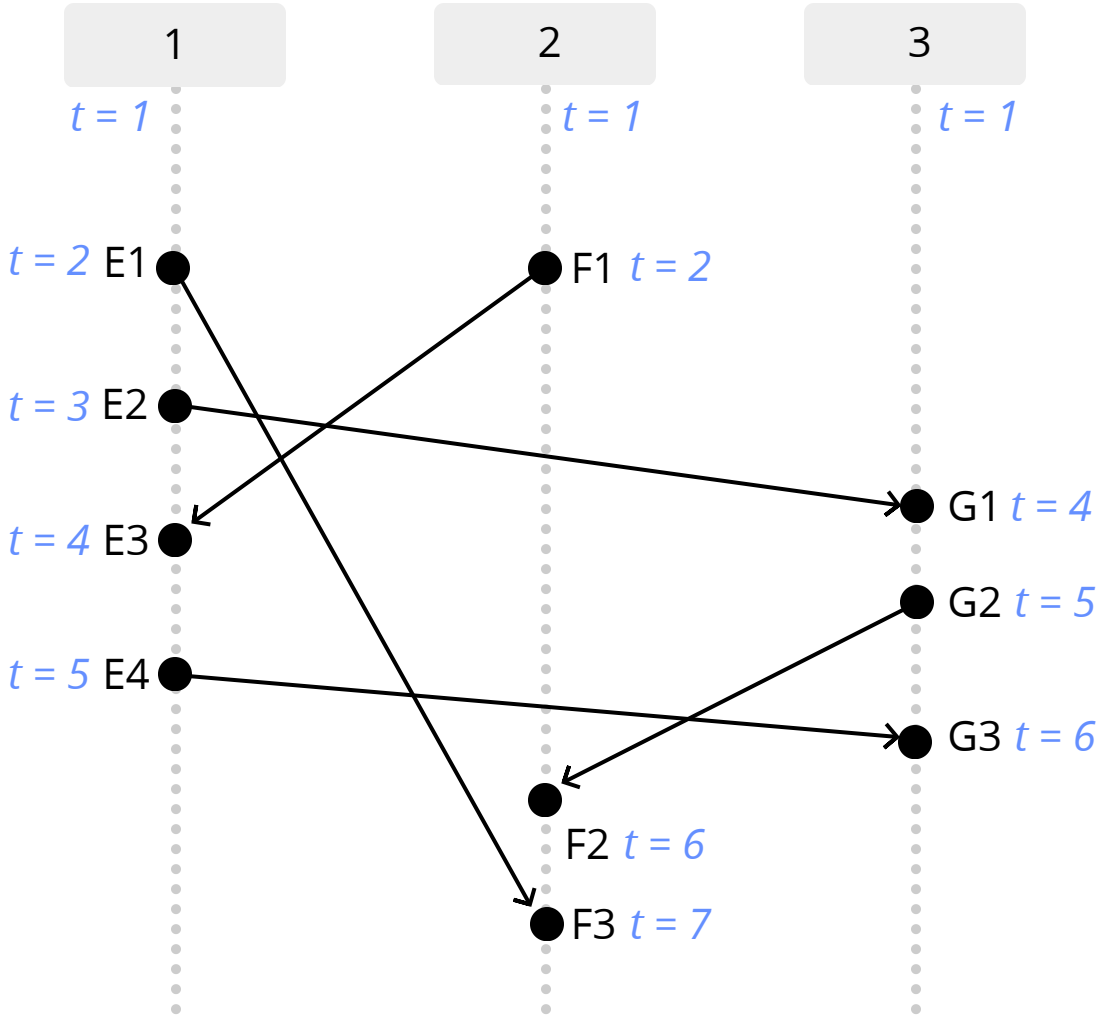
Exercice 1 - ordre total strict



Quel est l'ordre des événements ?

Exercice 1 - ordre total strict

Solution



Quel est l'ordre des événements ?

E1, F1, E2, E3, G1, E4, G2, F2, G3, F3

Exercice 2 - file d'attente distribuée

Plusieurs distributeurs de tickets, communiquant suivant le protocole de Lamport, où une demande de ticket représente un événement.

Chaque distributeur peut délivrer un ticket, affichant

- *la valeur actuelle du timestamp de Lamport*
- *et le numéro du distributeur qui l'a délivré.*

Chaque client doit d'abord obtenir un ticket au distributeur le plus proche.

Celui ayant le ticket (timestamp, numéro) le plus petit peut aller au guichet.

Questions :

1. Tout personne est-elle sûre de passer un jour ?
2. L'ordre d'arrivée des personnes est-il respecté ?
3. Quels avantages par rapport à un système central délivrant des numéros aux distributeurs ?
4. Que se passe-t-il si un distributeur tombe en panne et redémarre en repartant à son dernier timestamp ?

Exercice 2 - file d'attente distribuée

Solution

1. Tout personne est-elle sûre de passer un jour ?
   - Peut être prouvé par l'absurde : en notant que, si une personne ne passe pas, c'est qu'une autre a un ticket plus ancien et passe donc en premier, on voit que, si une personne ne passe jamais, c'est qu'il existe un nombre infini de personnes avec un ticket plus ancien, qui sont donc arrivées avant elles. Notre personne n'est donc jamais arrivée, une contradiction.
2. L'ordre d'arrivée des personnes est-il respecté ?
   - Pas toujours. Si une machine tombe en panne et reboot, son premier ticket sera le plus ancien. Sinon, si deux personnes arrivent presque au même moment sur des machines ayant le même timestamp, l'ordre sera arbitraire en fonction de l'id de la machine.
3. Quels avantages par rapport à un système central délivrant des numéros aux distributeurs ?
   - En cas de panne d'une machine, le reste fonctionne toujours, contrairement à un système centralisé qui est à l'arrêt si le processus central tombe en panne.
   - C'est aussi plus rapide s'il y a beaucoup de machines, car pas de bottleneck.
4. Que se passe-t-il si un distributeur tombe en panne et redémarre en repartant à son dernier timestamp ?
   - Comme dit à la question 2 ; le ticket délivré sera le plus ancien et la personne sera privilégiée.